



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

Organización de Lenguajes y Compiladores 1

Sección C

Escuela de Ingeniería de Ciencias Y Sistemas

Johnny Whillman Aldana Osorio

UNIDAD 9

Patrones de diseño y Patrón Intérprete

Aplicación con clase abstracta en TypeScript

Agenda



01 Patrones de diseño

Qué son y por qué ayudan a escribir software mantenible.

02 Clase abstracta

Base común para representar expresiones del lenguaje.

03 Patrón Intérprete

Estructura, participantes y relación con compiladores.

04 Ejemplo práctico

Mini intérprete de expresiones aritméticas en TypeScript.

05 Actividad

Extender el intérprete con una nueva operación.

Competencia que desarrollaremos

Marco formativo

Diseño orientado a objetos

Representar problemas mediante clases, herencia y responsabilidades claras.

Aplicación en compiladores

Usar el Patrón Intérprete para modelar una pequeña gramática y evaluar expresiones.



Al finalizar, el estudiante podrá identificar cuándo un patrón de comportamiento aporta orden al diseño y podrá construir una solución básica usando una clase abstracta como contrato común.

Responsabilidad

Diseñar software no es solo lograr que funcione: también debe ser claro, mantenible y entendible para otros.

Aplicación al laboratorio

Los patrones permiten justificar decisiones de diseño y evitar soluciones improvisadas o difíciles de modificar.

En esta sesión, la responsabilidad se refleja en escribir código organizado, explicar la intención de cada clase y respetar una estructura común para las expresiones.

Un patrón de diseño es una solución reutilizable para un problema común de diseño de software. No es una librería ni un bloque de código para copiar, sino una guía para organizar clases, objetos y responsabilidades.

Problema común

Aparece en muchos proyectos y contextos.

Solución probada

Propone una estructura clara y repetible.

Lenguaje común

Facilita discutir diseños con otros programadores.

Tipos de patrones de diseño

Conceptos base

Creacionales

Se enfocan en cómo crear objetos. Ejemplos: Factory Method, Builder, Singleton.

Estructurales

Organizan clases y objetos en estructuras flexibles. Ejemplos: Adapter, Facade, Composite.

Comportamiento

Definen cómo colaboran los objetos. Ejemplos: Strategy, Observer, Visitor, Interpreter.

El Patrón Intérprete pertenece a los patrones de comportamiento porque define cómo los objetos cooperan para evaluar una expresión.

Clase abstracta

Base del ejemplo

Una clase abstracta es una clase base que no se instancia directamente. Sirve para definir una estructura común que las clases hijas deben respetar.

- Puede incluir métodos ya implementados.
- Puede declarar métodos abstractos que las clases hijas deben implementar.
- Funciona como contrato común dentro de una jerarquía de clases.
- En TypeScript se declara con la palabra reservada `abstract`.

Clase abstracta en TypeScript

Ejemplo base

```
abstract class Expresion {  
  abstract interpretar(contexto: Map<string,  
    number>): number;  
  
  describir(): string {  
    return "Nodo de una expresion";  
  }  
}
```

Expresion será la base de todo el mini lenguaje. Cada nodo del árbol deberá saber

Punto importante

No se crea new Expresion(); se crean clases concretas que heredan de ella.

Patrón Intérprete

Definición

El Patrón Intérprete define una representación para la gramática de un lenguaje y utiliza esa representación para interpretar oraciones válidas dentro de ese lenguaje.

Idea central

Cada regla importante del lenguaje se convierte en una clase.

Resultado

Los objetos se combinan para formar un árbol que puede evaluarse.

En compiladores, esta idea se relaciona con el AST: cada nodo representa una parte del programa o expresión.

Participantes del patrón

Patrón Intérprete

- `AbstractExpression`: define el método `interpretar()`.
- `TerminalExpression`: representa elementos simples, como números o variables.
- `NonTerminalExpression`: combina otras expresiones, como suma o multiplicación.
- `Context`: guarda información externa necesaria para interpretar.
- `Client`: construye el árbol y solicita la interpretación.

Relación con compiladores

De gramática a objetos

Entrada

Una cadena escrita en un lenguaje definido, por ejemplo:
 $x + 5 * y$.

Estructura

El parser puede producir un árbol de expresiones o AST.

Interpretación

Cada nodo del árbol sabe cómo evaluarse mediante `interpretar()`.

Por esta razón, el patrón es útil en calculadoras, reglas de negocio, filtros, validadores y lenguajes pequeños de dominio específico.

Ejemplo práctico



Construiremos un intérprete para evaluar expresiones con números, variables, suma y multiplicación.

Expresión

$(x + 5) * y$

Contexto

$x = 2, y = 3$

Resultado esperado: 21

Gramática simplificada

Ejemplo práctico

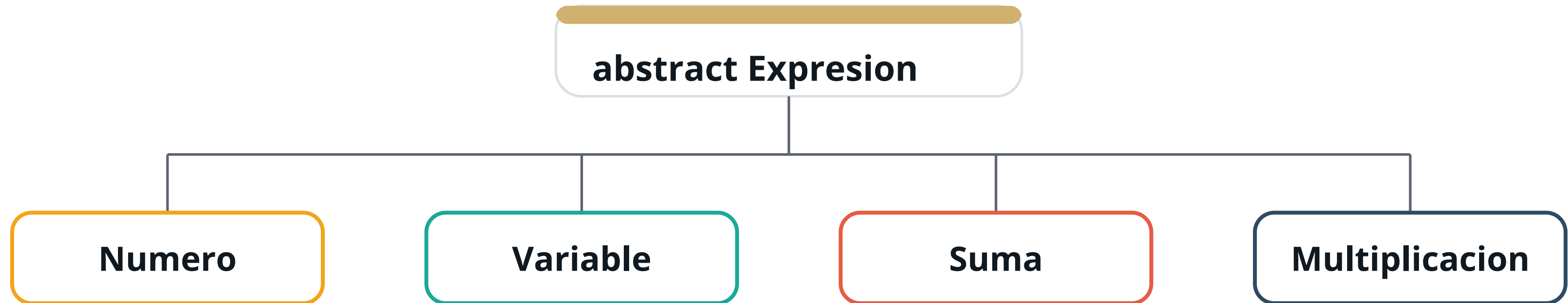
```
EXPRESION  -> NUMERO
            | VARIABLE
            | SUMA
            | MULTIPLICACION

SUMA        -> EXPRESION + EXPRESION
MULT        -> EXPRESION * EXPRESION
```

La clase abstracta `Expresion` representa el concepto general. Las clases `Numero`, `Variable`, `Suma` y `Multiplicacion` representan casos concretos del lenguaje.

Estructura conceptual

Clase abstracta + herencia



Todas las clases concretas heredan de Expresion y, por lo tanto, deben implementar interpretar(contexto).

Expresiones terminales

Número y variable

```
class Numero extends Expresion {  
    constructor(private valor: number) { super(); }  
  
    interpretar(): number {  
        return this.valor;  
    }  
}  
  
class Variable extends Expresion {  
    constructor(private nombre: string) { super(); }  
  
    interpretar(ctx: Map<string, number>): number {  
        return ctx.get(this.nombre) ?? 0;  
    }  
}
```

Una expresión terminal ya no se divide en más expresiones. Numero devuelve su valor y Variable consulta su dato en el contexto.

Expresiones no terminales

Suma y multiplicación

```
class Suma extends Expresion {
    constructor(private izq: Expresion,
                private der: Expresion) { super(); }

    interpretar(ctx: Map<string, number>): number {
        return this.izq.interpretar(ctx) + this.der.interpretar(ctx);
    }
}

class Multiplicacion extends Expresion {
    constructor(private izq: Expresion,
                private der: Expresion) { super(); }

    interpretar(ctx: Map<string, number>): number {
        return this.izq.interpretar(ctx) * this.der.interpretar(ctx);
    }
}
```

Una expresión no terminal combina otras expresiones. Por eso el patrón funciona naturalmente como un árbol.

Construcción e interpretación

Uso del árbol

```
const contexto = new Map<string, number>();
contexto.set("x", 2);
contexto.set("y", 3);

const expresion = new Multiplicacion(
  new Suma(new Variable("x"), new Numero(5)),
  new Variable("y")
);

console.log(expresion.interpretar(contexto)); // 21
```

El cliente crea el árbol manualmente o lo recibe desde un parser. Luego solo llama interpretar() desde la raíz.

Ventajas y precauciones

Uso recomendado

Ventajas

Organiza reglas del lenguaje en clases claras y facilita extender el lenguaje con nuevas expresiones.

Precauciones

Si el lenguaje crece demasiado, puede generar muchas clases y conviene usar un parser más robusto.

Úsalo cuando el lenguaje sea pequeño, estable y fácil de representar como árbol.

Conceptos clave aprendidos

Cierre de teoría

- Un patrón de diseño es una solución reutilizable a un problema común.
- Una clase abstracta define una base común y obliga a implementar métodos clave.
- El Patrón Intérprete representa reglas de un lenguaje como clases.
- El método interpretar() se ejecuta recursivamente sobre un árbol de expresiones.
- El contexto almacena información externa, como valores de variables.

- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software.
- Aho, A., Lam, M., Sethi, R. y Ullman, J. Compilers: Principles, Techniques & Tools.
- Loudon, K. Lenguajes de programación: principios y práctica.
- TypeScript Handbook: clases y clases abstractas.
- Programa de Laboratorio de Lenguajes Formales y de Programación, USAC.

Practiquemos lo aprendido



Reto individual o en parejas: extender el intérprete agregando una nueva operación.

Opción A

Crear la clase Resta.

Opción B

Crear la clase Division.

Actividad de refuerzo

Extensión del intérprete

- Crear una clase Resta o Division que herede de Expresion.
- Recibir dos expresiones en el constructor: izquierda y derecha.
- Implementar interpretar(contexto) usando los resultados de sus hijos.
- Probar la expresión: $(x + 5) - y$ o $(x + 5) / y$.
- Explicar qué participante del patrón representa la clase creada.



¿DUDAS?



**¡GRACIAS POR SU
ATENCIÓN!**

Recuerda utilizar el foro semanal para consultar cualquier duda que surja durante la semana.